

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.9

FILTER

 Copy as Markdown

Filter

Filters narrow which rows a query returns. You build them in the `where` callback that most database methods accept, and they are statically type-checked. The callback receives a table descriptor, by convention named `t`, which exposes each column of the table as a typed field. Filtering happens in the database, so only the matching rows are sent to the server.

This page covers filtering on column values, on logical combinations, and on related rows. To include related data in the result instead of filtering by it, see [Relation queries](#).

Column operations

Each column datatype supports the set of operations that makes sense for that type.

Equals

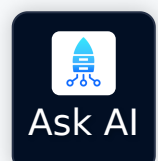
Compare a column to an exact value, meaning only rows that match exactly will remain in the result.

```
await User.db.find(
  session,
  where: (t) => t.name.equals('Alice'),
);
```

In the example we fetch all users with the name Alice.

Not equals is the negated version of equals.

```
await User.db.find(
  session,
  where: (t) => t.name.notEquals('Bob')
);
```



In the example we fetch all users with a name that is not Bob. If a non-`null` value is used as an argument for the `notEquals` comparison, rows with a `null` value in the column will be included in the result.

Comparison operators

The `>`, `≥`, `<`, and `≤` operators compare a column to a value. They are supported for `int`, `double`, `Duration`, `DateTime`, `String`, and `UuidValue` columns.

```
await User.db.find(
  session,
  where: (t) => t.age > 25
);
```

In the example we fetch all users that are older than 25.

INFO

Comparison operators check value types at runtime. Comparing a `double` column against an `int` literal, such as `t.price > 500`, compiles but throws. Write the literal to match the column type, `t.price > 500.0`.

Between

The `between` method takes two values and checks if the column's value is between them, inclusive of the boundaries. It is available on `int`, `double`, `Duration`, and `DateTime` columns.

```
await User.db.find(
  session,
  where: (t) => t.age.between(18, 65)
);
```

In the example we fetch all users between 18 and 65 years old. This can also be expressed as `(t.age ≥ 18) & (t.age ≤ 65)`.

The `notBetween` operation negates the condition. The boundaries are still inclusive.

```
await User.db.find(
  session,
  where: (t) => t.age.notBetween(18, 65)
);
```

In the example we fetch all users that are not between 18 and 65 years old. This can also be expressed as `(t.age < 18) | (t.age > 65)`.

In set

In set can be used to match with several values at once. The `inSet` method makes an exact comparison, like equals, but against every value in the set.

```
await User.db.find(
  session,
  where: (t) => t.name.inSet({'Alice', 'Bob'})
);
```

In the example we fetch all users with a name matching either Alice or Bob. If an empty set is used as an argument for the `inSet` comparison, no rows will be included in the result.

The `notInSet` operation negates the condition.

```
await User.db.find(
  session,
  where: (t) => t.name.notInSet({'Alice', 'Bob'})
);
```

In the example we fetch all users with a name not matching Alice or Bob. Rows with a `null` value in the column will be included in the result. If an empty set is used as an argument for the `notInSet` comparison, all rows will be included in the result.

Like

Like performs pattern matching against `String` columns. The matcher is case-sensitive.

Two special characters match partial entries:

- `%` matches any sequence of characters.
- `_` matches any single character.

String	Matcher	Is matching
abc	a%	true
abc	_b%	true
abc	a_c	true
abc	b_	false

We use like to match against a partial string.

```
await User.db.find(
  session,
  where: (t) => t.name.like('A%')
);
```

In the example we fetch all users with a name that starts with A.

There is a negated version of like that can be used to exclude rows from the result.

```
await User.db.find(
  session,
  where: (t) => t.name.notLike('B%')
);
```

In the example we fetch all users with a name that does not start with B.

Ilike

The `ilike` method works the same as `like` but is case-insensitive.

```
await User.db.find(
  session,
  where: (t) => t.name.ilike('a%')
);
```

In the example we fetch all users with a name that starts with a or A.

There is a negated version, `notIlike`, that can be used to exclude rows from the result.

```
await User.db.find(
  session,
  where: (t) => t.name.notIlike('b%')
);
```

In the example we fetch all users with a name that does not start with b or B.

INFO

On SQLite, both `like` and `ilike` use SQLite's `LIKE` operator, which is case-insensitive for ASCII characters. This means `like` is not case-sensitive on SQLite the way it is on Postgres.

Logical operators

Logical operators are also supported when filtering, allowing you to chain multiple statements together to create more complex queries.

The `&` operator is used to chain two statements together with an `and` operation.

```
await User.db.find(
  session,
  where: (t) => (t.name.equals('Alice') & (t.age > 25))
);
```

In the example we fetch all users with the name "Alice" *and* are older than 25.

The `|` operator is used to chain two statements together with an `or` operation.

```
await User.db.find(
  session,
  where: (t) => (t.name.like('A%') | t.name.like('B%'))
);
```

In the example we fetch all users that have a name that starts with A *or* B.

The `~` operator is used to negate an expression with a `not` operation.

```
await User.db.find(
  session,
  where: (t) => ~t.name.equals('Alice')
);
```

In the example we fetch all users that do *not* have the name "Alice".

The `~` operator can also be used with more complex expressions:

```
await User.db.find(
  session,
  where: (t) => ~(t.name.like('A%') | (t.age > 25))
);
```

In the example we fetch all users that do *not* have a name starting with "A" *and* are *not* older than 25.

Place each operand within parentheses, as in the examples above. Dart's operator precedence binds `&` and `|` tighter than comparisons, so an expression like `t.name.like('A%') | t.age > 25` does not compile without the parentheses around `(t.age > 25)`.

Vector distance operators

All vector field types support specialized distance operations for similarity search. Available vector distance operations:

Vector, HalfVector, and SparseVector fields:

- `distanceL2` - Euclidean (L2) distance.
- `distanceInnerProduct` - Inner product distance.
- `distanceCosine` - Cosine distance.
- `distanceL1` - Manhattan or taxicab (L1) distance.

Bit vector fields:

- `distanceHamming` - Hamming distance.
- `distanceJaccard` - Jaccard distance.

A distance operation returns a numeric value, so you can compare it in `where`, order by it, and combine it with other filters. The canonical similarity search orders by distance and limits the result:

```
// The vector to compare against.
var queryVector = Vector([0.1, 0.2, 0.3, ...]);

// Find the ten documents most similar to the query vector.
var similarDocs = await Document.db.find(
  session,
  where: (t) => t.embedding.distanceCosine(queryVector) < 0.5,
  orderBy: (t) => t.embedding.distanceCosine(queryVector),
  limit: 10,
);
```

The same shape works for every distance operation, such as `distanceInnerProduct` against a `SparseVector` query or `distanceHamming` against a `Bit` query. The distance can also be range-filtered or combined with regular column filters:

```
// Filter by distance range.
var mediumSimilarity = await Document.db.find(
  session,
  where: (t) => t.embedding.distanceL2(queryVector).between(0.3, 0.8),
);
```

```
// Combine with other filters.
var filteredSimilarity = await Document.db.find(
  session,
  where: (t) => t.category.equals('article') &
    (t.embedding.distanceCosine(queryVector) < 0.7),
  orderBy: (t) => t.embedding.distanceCosine(queryVector),
  limit: 10,
);
```

TIP

For optimal performance with vector similarity searches, consider creating specialized vector indexes (HNSW or IVFFLAT) on your vector fields. See the [Vector indexes](#) section for more details. To fine-tune query execution, you can also set appropriate [runtime parameters](#) for vector queries.

Relation operations

When two models are connected with a [one-to-one](#) or [one-to-many](#) object relation, you can filter on the related rows directly.

One-to-one

For 1:1 relations the columns of the relation can be accessed directly on the relation field. This enables filtering on related objects properties.

```
await User.db.find(
  session,
  where: (t) => t.address.street.like('%road%')
);
```

In the example each user has a relation to an address with a street field. The query fetches all users whose related address has a street containing the word "road".

One-to-many

For 1:n relations, special filter methods let you filter on properties of the related rows as a group: how many there are, whether any or none exist, or whether all of them match a condition.

Count

Count can be used to count the number of related entries in a 1:n relation. The `count` always needs to be compared with a constant value.

```
await User.db.find(
```

```
session,  
where: (t) => t.orders.count() > 3  
);
```

In the example we fetch all users with more than three orders.

We can apply a sub-filter to the **count** operator to filter the related entries before they are counted.

```
await User.db.find(  
  session,  
  where: (t) => t.orders.count((o) => o.itemType.equals('book')) > 3  
);
```

In the example we fetch all users with more than three "book" orders.

None

None can be used to retrieve rows that have no related entries in a 1:n relation. If a related entry exists, the row is omitted from the result.

```
await User.db.find(  
  session,  
  where: (t) => t.orders.none()  
);
```

In the example we fetch all users that have no orders.

We can apply a sub-filter to the **none** operator to filter the related entries. If any related entry matches the sub-filter, the row is omitted from the result.

```
await User.db.find(  
  session,  
  where: (t) => t.orders.none((o) => o.itemType.equals('book'))  
);
```

In the example we fetch all users that have no "book" orders.

Any

Any works similarly to the **any** method on lists in Dart. If there exists any related entry then include the row in the result.

```
await User.db.find(  
  session,
```

```
where: (t) => t.orders.any()  
);
```

In the example we fetch all users that have any order.

We can apply a sub-filter to the **any** operator to filter the related entries. If any related entry matches the sub-filter, the row is included in the result.

```
await User.db.find(  
  session,  
  where: (t) => t.orders.any((o) => o.itemType.equals('book'))  
);
```

In the example we fetch all users that have any "book" order.

Every

Every works similarly to the **every** method on lists in Dart. If every related entry matches the sub-filter then include the row in the result. For the **every** operator the sub-filter is mandatory.

```
await User.db.find(  
  session,  
  where: (t) => t.orders.every((o) => o.itemType.equals('book'))  
);
```

In the example we fetch all users that have only "book" orders.

Geography operators

All geography field types (**GeographyPoint**, **GeographyLineString**, **GeographyPolygon**, **GeographyGeometryCollection**) support spatial filter and ordering operations.

intersects

Returns rows where the geography column spatially intersects the given geography value. Wraps **ST_Intersects**.

```
var point = GeographyPoint(longitude: 2.35, latitude: 48.85);  
  
await Store.db.find(  
  session,  
  where: (t) => t.location.intersects(point),  
);
```

distanceWithin

Returns rows where the geography column is within a given distance (in metres) of the given geography value. Wraps `ST_DWithin`.

```
var point = GeographyPoint(longitude: 2.35, latitude: 48.85);

await Store.db.find(
  session,
  where: (t) => t.location.distanceWithin(point, 1000),
);
```

distance

Returns the distance in metres between the geography column and the given geography value. Wraps `ST_Distance`. The result can be used in `orderBy` for nearest-first ordering, or compared numerically in `where`.

```
var point = GeographyPoint(longitude: 2.35, latitude: 48.85);

// Order results nearest-first
await Store.db.find(
  session,
  orderBy: (t) => t.location.distance(point),
);

// Filter by distance and order results nearest-first
await Store.db.find(
  session,
  where: (t) => t.location.distance(point) < 5000,
  orderBy: (t) => t.location.distance(point),
);
```

contains

Returns rows where the geography column fully contains the given geography value. Wraps `ST_Covers`.

```
var point = GeographyPoint(longitude: 2.35, latitude: 48.85);

await DeliveryZone.db.find(
  session,
  where: (t) => t.boundary.contains(point),
);
```

within

Returns rows where the geography column is fully within the given geography value. Wraps `ST_CoveredBy`.

```
var zone = GeographyPolygon(
```

```
    exteriorRing: [  
      GeographyPoint(longitude: 2.30, latitude: 48.82),  
      GeographyPoint(longitude: 2.40, latitude: 48.82),  
      GeographyPoint(longitude: 2.40, latitude: 48.90),  
      GeographyPoint(longitude: 2.30, latitude: 48.90),  
      GeographyPoint(longitude: 2.30, latitude: 48.82),  
    ],  
  );  
  
await Store.db.find(  
  session,  
  where: (t) => t.location.within(zone),  
);
```

TIP

For optimal performance with spatial queries, consider creating a spatial index on your geography fields. See the [Geography indexes](#) section for more details.